

UNIVERSIDAD TECNOLÓGICA DE HONDURAS (UTH)



PROGRAMACIÓN ORIENTADA A OBJETOS

INFORME TÉCNICO

Nombre del Proyecto: Sistema de Gestión Organizacional en Java (POO + SQLite + Swing)

Docente: Ing. Ricardo Lagos

Integrantes:

1. Audrey Azucena Castro Rodriguez	Cta.: 202510030095
2. Jessy Odalis Villeda Obando	Cta.: 202510120041
3. Dany Alberto Cruz Matute	Cta.: 201830060173
4. Hendric Jahir Castellón Arriaga	Cta.: 202520050074
5. Jireh Adonay Aguilar Madrid	Cta.: 202320030011
6. Alejandro Ivan Machado Garcia	Cta.: 202530060207
7. Carlos Leonel Martinez Oliva	Cta.: 202510030095
8. Jefry Yosseth Vásquez Ceballos	Cta.: 202120120003

03 de agosto del 2026



INFORME TÉCNICO — PROYECTO FINAL

Programación Orientada a Objetos

“Sistema de Gestión Organizacional”

GRUPO#3

Aplicación de escritorio desarrollada en Java (Swing + SQLite + JDBC)

Tabla de contenido

1. Introducción	5
2. Objetivos	6
2.1 Objetivo general	6
2.2 Objetivos específicos	6
3. Alcance del proyecto	7
3.1 Incluye	7
3.2 No incluye	7
4. Marco teórico	8
4.1 Clase y objeto	8
4.2 Encapsulamiento	8
4.3 Abstracción	8
4.4 Herencia	8
4.5 Polimorfismo	8
4.6 Asociación, agregación y composición	8
4.7 Arquitectura en capas y patrón MVC	9
4.8 Patrón Singleton	9
5. Metodología utilizada	10
5.1 Fases de desarrollo	10
5.2 Organización del equipo	10
6. Desarrollo del sistema	11
6.1 Arquitectura del sistema	11
6.1.1 Descripción de las capas	11
6.2 Tecnologías utilizadas	12
6.3 Diagramas del sistema	12
6.3.1 Diagrama de clases (UML)	12
6.3.2 Diagrama entidad-relación	13
6.4 Estructura del modelo de clases	14
6.5 Módulos funcionales de la aplicación	14
6.6 Persistencia de datos	14
6.7 Validaciones y manejo de excepciones	15
7. Resultados obtenidos	16
8. Conclusiones	19
9. Recomendaciones	20
Anexo A. Instrucciones para la ejecución del sistema	21
A.1 Requisitos previos	21

A.2 Ejecución a partir del código fuente	21
A.3 Primer inicio de sesión	21
A.4 Uso general	21

1. Introducción

La gestión eficiente de la información organizacional —empleados, departamentos, cargos, países de operación y proyectos— es una necesidad común a cualquier empresa, independientemente de su tamaño. Cuando esta información se administra de forma manual o dispersa en hojas de cálculo, resulta difícil mantener la integridad de los datos, controlar las relaciones entre entidades (por ejemplo, qué empleado pertenece a qué departamento o participa en qué proyecto) y generar reportes confiables.

El presente informe documenta el desarrollo del proyecto final de la asignatura de Programación Orientada a Objetos (POO): un Sistema de Gestión Organizacional construido como aplicación de escritorio en Java. El proyecto tiene un doble propósito: por un lado, resolver un problema real de administración de datos empresariales mediante operaciones CRUD (crear, leer, actualizar y eliminar) completas; y por otro, servir como caso de estudio práctico para aplicar de forma correcta y verificable los pilares de la Programación Orientada a Objetos — encapsulamiento, abstracción, herencia, polimorfismo, asociación, agregación y composición — dentro de una arquitectura en capas.

El sistema fue construido en Java SE, utiliza Swing con una interfaz de tipo MDI (Multiple Document Interface) para la experiencia de usuario, persiste la información en una base de datos SQLite mediante JDBC, e incorpora funcionalidades adicionales como control de acceso por usuario y exportación de reportes a formatos CSV y PDF. A lo largo de este documento se describen los objetivos, el alcance, el marco teórico, la metodología de trabajo, la arquitectura y las tecnologías empleadas, los resultados obtenidos y las conclusiones del equipo de desarrollo.

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación de escritorio en Java, denominada Sistema de Gestión Organizacional, que permita administrar de forma integral la estructura de una empresa —países, departamentos, cargos, empleados, proyectos y asignaciones—, aplicando de manera consistente los principios de la Programación Orientada a Objetos y garantizando la persistencia de la información mediante una base de datos relacional.

2.2 Objetivos específicos

- Modelar mediante clases las entidades reales del dominio empresarial (Persona, Empleado, País, Departamento, Cargo, Proyecto, Asignación y Empresa), aplicando encapsulamiento estricto con atributos privados y acceso mediante métodos getter y setter.
- Aplicar herencia y polimorfismo definiendo una clase abstracta Persona de la cual deriva Empleado, sobrescribiendo métodos como `getDescripcion()` y `toString()` para demostrar comportamiento polimórfico.
- Implementar relaciones de asociación, agregación y composición entre objetos, incluyendo una relación muchos a muchos (N:M) entre Empleado y Proyecto resuelta mediante la clase intermedia Asignacion.
- Construir una arquitectura en capas (Modelo – DAO – Controlador – Vista – Conexión) que separe claramente las responsabilidades del sistema y facilite su mantenimiento.
- Implementar operaciones CRUD completas para cada entidad, utilizando JDBC con `PreparedStatement` y una base de datos SQLite con integridad referencial mediante llaves foráneas.
- Desarrollar una interfaz gráfica funcional y moderna con Swing, empleando el patrón MDI (`JDesktopPane` / `JInternalFrame`), control de acceso mediante inicio de sesión y exportación de datos a CSV y PDF.

3. Alcance del proyecto

El sistema desarrollado cubre la gestión completa de la estructura organizacional de una empresa a nivel de aplicación de escritorio monousuario, con persistencia local en SQLite. A continuación se detalla lo que el proyecto incluye y lo que queda fuera de su alcance.

3.1 Incluye

- Modelado orientado a objetos completo de las entidades del dominio (Persona, Empleado, País, Departamento, Cargo, Proyecto, Asignación, Empresa).
- Persistencia de datos en SQLite mediante JDBC, con creación automática del esquema al iniciar la aplicación.
- Interfaz gráfica funcional bajo el patrón MDI, con un módulo independiente por entidad y un panel de control (Dashboard) con indicadores generales.
- Operaciones CRUD (registrar, listar, modificar y eliminar) para países, departamentos, cargos, empleados, proyectos y asignaciones.
- Módulo de autenticación de usuario (inicio de sesión) para el ingreso al sistema.
- Exportación de la información mostrada en tablas a formatos CSV y PDF.
- Validaciones de campos obligatorios y manejo estructurado de excepciones de negocio (ValidacionException).
- Documentación técnica, manual de usuario y diagramas de diseño (clases y entidad-relación).

3.2 No incluye

- Operación en red o modo multiusuario concurrente; el sistema está diseñado para ejecutarse de forma local.
- Roles y permisos diferenciados más allá de la autenticación básica de acceso.
- Integración con servicios o APIs externas.
- Reportes estadísticos avanzados más allá de los indicadores del panel de control y las exportaciones CSV/PDF.
- Migración o soporte para motores de base de datos distintos a SQLite.

4. Marco teórico

La Programación Orientada a Objetos (POO) es un paradigma de desarrollo de software que organiza el código en torno a “objetos”: entidades que combinan estado (atributos) y comportamiento (métodos). A continuación se describen los conceptos fundamentales de POO y cómo se interpretan conceptualmente dentro del dominio del proyecto.

4.1 Clase y objeto

Una clase es una plantilla que define la estructura y el comportamiento común de un conjunto de objetos; un objeto es una instancia concreta de una clase, con valores propios en sus atributos. En el proyecto, clases como Empleado, Departamento o Proyecto definen la estructura de los registros administrados, mientras que cada fila almacenada en la base de datos corresponde a una instancia de esa clase en memoria.

4.2 Encapsulamiento

Consiste en ocultar el estado interno de un objeto y exponer únicamente lo necesario a través de una interfaz pública (métodos). Todas las clases del modelo declaran sus atributos como privados y ofrecen métodos getter y setter públicos para consultarlos o modificarlos, evitando el acceso directo e incontrolado a los datos.

4.3 Abstracción

Es la capacidad de representar los aspectos esenciales de una entidad del mundo real dentro del software, ignorando los detalles irrelevantes para el problema que se resuelve. Cada clase del modelo (Pais, Cargo, Departamento, Proyecto, Empleado) representa de forma simplificada una entidad real de la organización, con solo los atributos relevantes para el sistema.

4.4 Herencia

Permite que una clase (subclase) derive de otra (superclase), reutilizando y extendiendo su comportamiento. En el proyecto se define la clase abstracta Persona, que agrupa los atributos y comportamientos comunes a cualquier persona del sistema (id, nombre, apellido); la clase Empleado extiende de Persona y añade los atributos propios de un trabajador (salario, país, departamento y cargo).

4.5 Polimorfismo

Es la capacidad de que distintas clases respondan de forma diferente a un mismo mensaje o método. Persona declara el método abstracto getDescripcion(), y Empleado ofrece su propia implementación; asimismo, el método toString() se sobrescribe en varias clases del modelo para proveer una representación textual adecuada a cada entidad.

4.6 Asociación, agregación y composición

- Asociación: relación entre objetos independientes entre sí. Empleado está asociado a Pais, Departamento y Cargo mediante referencias, pero estas entidades existen de forma independiente al empleado.

- Composición: relación de pertenencia fuerte, donde el objeto contenedor administra el ciclo de vida del contenido. La clase Empresa contiene y administra las colecciones de Departamentos, Empleados y Proyectos.
- Relación muchos a muchos (N:M): se da entre Empleado y Proyecto, ya que un empleado puede participar en varios proyectos y un proyecto puede tener varios empleados asignados; esta relación se resuelve mediante la clase intermedia Asignacion, que además añade el atributo horasAsignadas.

4.7 Arquitectura en capas y patrón MVC

Es un estilo de diseño de software que separa una aplicación en niveles de responsabilidad independientes (por ejemplo: presentación, lógica de negocio y acceso a datos), de modo que los cambios en una capa afecten mínimamente a las demás. El proyecto adapta este principio mediante una variante del patrón Modelo–Vista–Controlador (MVC), incorporando además una capa DAO (Data Access Object) dedicada exclusivamente al acceso a la base de datos.

4.8 Patrón Singleton

Es un patrón de diseño creacional que garantiza que una clase tenga una única instancia accesible globalmente. La clase ConexionSQLite implementa este patrón para asegurar que exista una sola conexión activa hacia la base de datos durante la ejecución de la aplicación.

5. Metodología utilizada

El equipo de trabajo adoptó un enfoque iterativo e incremental, inspirado en prácticas ágiles, que permitió construir el sistema en etapas cortas y verificables en lugar de intentar completarlo en un único bloque de desarrollo. Este enfoque facilitó la coordinación entre los ocho integrantes del equipo y permitió corregir errores de diseño en fases tempranas del proyecto.

5.1 Fases de desarrollo

- **Análisis de requisitos:** identificación de las entidades del dominio (empleados, departamentos, cargos, países, proyectos) y de las operaciones que el sistema debía soportar.
- **Diseño orientado a objetos:** elaboración del diagrama de clases, definición de atributos, relaciones (asociación, composición, herencia) y responsabilidades de cada clase.
- **Diseño de la base de datos:** definición del modelo entidad-relación, las tablas, llaves primarias y foráneas, y las restricciones de integridad referencial en SQLite.
- **Implementación por capas:** desarrollo del modelo, luego la capa de acceso a datos (DAO), después los controladores con las validaciones de negocio y, finalmente, las vistas en Swing.
- **Pruebas funcionales:** verificación manual de cada operación CRUD, de las validaciones de campos obligatorios y de la integridad de las relaciones entre entidades.
- **Integración y control de versiones:** unificación del código de los distintos integrantes mediante un repositorio compartido, resolviendo conflictos de fusión y validando que el sistema compilara y ejecutara correctamente.
- **Documentación:** elaboración del informe técnico, el manual de usuario y los diagramas finales del sistema.

5.2 Organización del equipo

El equipo, integrado por ocho estudiantes, distribuyó el trabajo por módulo funcional (Países, Departamentos, Cargos, Empleados, Proyectos, Asignaciones, Autenticación/Dashboard y Exportación de reportes), de modo que cada integrante fue responsable del ciclo completo de su módulo —modelo, DAO, controlador y vista— antes de integrarlo al resto del sistema. Las decisiones de diseño transversales (paquetes, convenciones de nombres, estilo visual común y esquema de base de datos) se acordaron de forma conjunta al inicio del proyecto para mantener la coherencia entre los módulos desarrollados por distintos integrantes.

6. Desarrollo del sistema

6.1 Arquitectura del sistema

El sistema se organiza en cinco paquetes que representan capas de responsabilidad claramente diferenciadas, siguiendo una variante del patrón Modelo–Vista–Controlador (MVC) enriquecida con una capa DAO dedicada al acceso a datos. Esta separación permite que los cambios en la interfaz gráfica no afecten la lógica de negocio, y que los cambios en la forma de persistir los datos no afecten al modelo ni a las vistas.

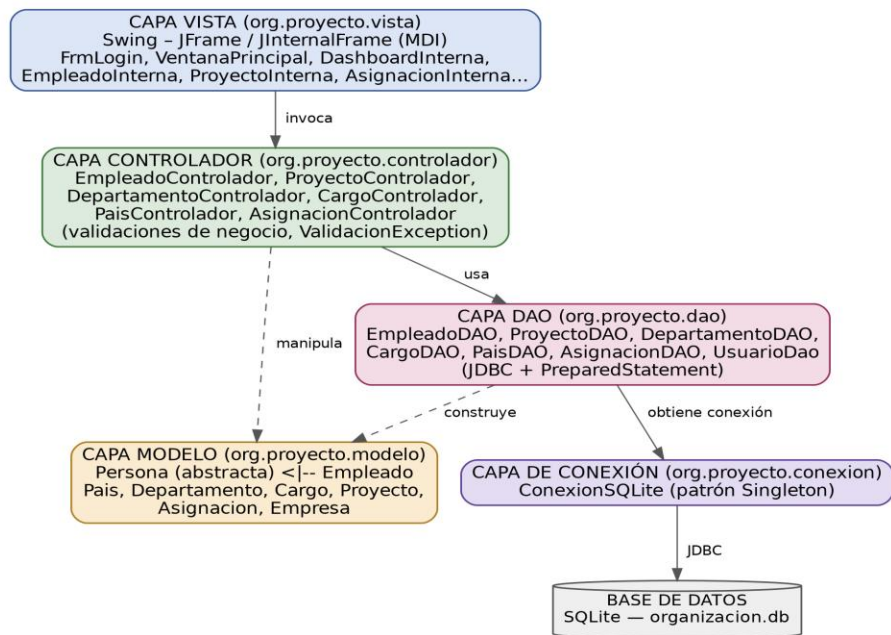


Figura 1. Arquitectura en capas del Sistema de Gestión Organizacional.

6.1.1 Descripción de las capas

- **org.proyecto.modelo** — Contiene las clases de dominio (Persona, Empleado, Pais, Departamento, Cargo, Proyecto, Asignacion, Empresa) que representan los datos y las reglas propias de cada entidad.
- **org.proyecto.dao** — Contiene las clases de acceso a datos (EmpleadoDAO, ProyectoDAO, DepartamentoDAO, CargoDAO, PaisDAO, AsignacionDAO, UsuarioDao), responsables de traducir operaciones sobre objetos en sentencias SQL mediante JDBC.
- **org.proyecto.controlador** — Contiene los controladores (EmpleadoControlador, ProyectoControlador, DepartamentoControlador, CargoControlador, PaisControlador, AsignacionControlador) que validan los datos antes de delegarlos a los DAO y encapsulan la lógica de negocio, lanzando ValidacionException cuando corresponde.
- **org.proyecto.vista** — Contiene las ventanas Swing: FrmLogin (autenticación), VentanaPrincipal (contenedor MDI con JDesktopPane), DashboardInterna (panel de indicadores) y una ventana interna (JInternalFrame) por cada módulo de gestión, además de utilidades de exportación (ExportadorCSV, ExportadorPDF) y de estilo (EstiloUI, UtilidadesUI).
- **org.proyecto.conexion** — Contiene ConexionSQLite, clase Singleton responsable de abrir la conexión JDBC hacia SQLite, habilitar las llaves foráneas y crear el esquema de la base de datos si no existe.

6.2 Tecnologías utilizadas

Tecnología / Herramienta	Uso dentro del proyecto
Java SE 17	Lenguaje y plataforma principal de desarrollo del sistema.
Swing (javax.swing)	Construcción de la interfaz gráfica de escritorio, bajo un esquema MDI (JDesktopPane / JInternalFrame).
SQLite	Motor de base de datos relacional embebido, utilizado para la persistencia local de la información.
JDBC (sqlite-jdbc 3.45.1.0)	Driver de conexión entre la aplicación Java y la base de datos SQLite.
iText 5 (itextpdf 5.5.13.3)	Generación de reportes en formato PDF a partir del contenido de las tablas del sistema.
Apache Maven	Gestión de dependencias y empaquetado del proyecto en un archivo .jar ejecutable (maven-shade-plugin).
Git / GitHub	Control de versiones y repositorio colaborativo para la integración del código de los ocho integrantes.

6.3 Diagramas del sistema

6.3.1 Diagrama de clases (UML)

El siguiente diagrama resume las clases del paquete modelo y las relaciones de herencia, asociación y composición implementadas en el sistema.

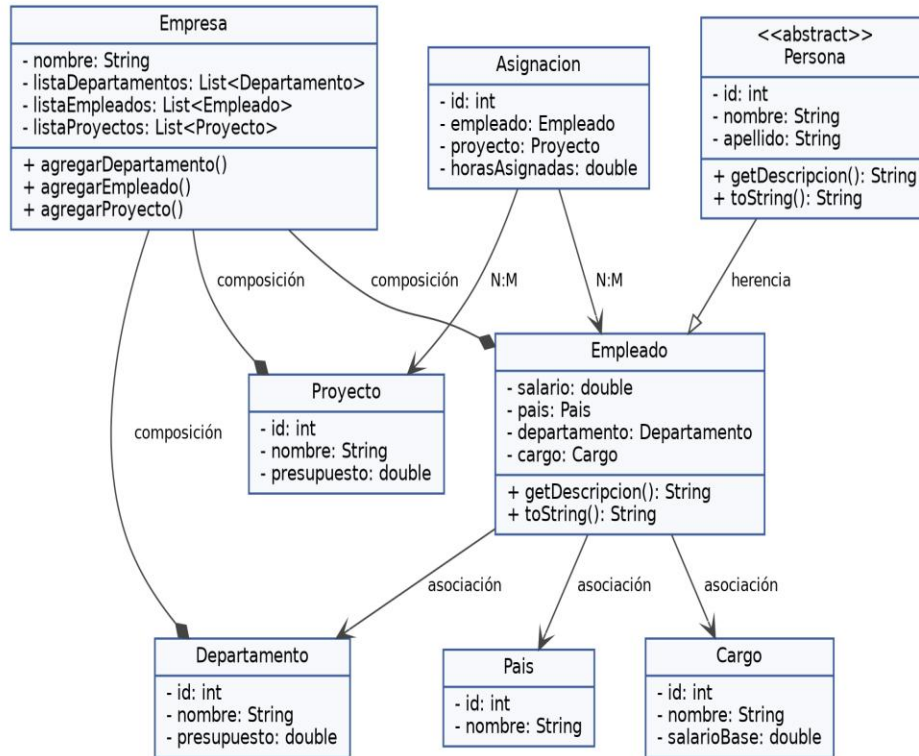


Figura 2. Diagrama de clases del dominio.

6.3.2 Diagrama entidad-relación

El modelo de base de datos refleja fielmente el modelo de clases del dominio. Las tablas pais, departamento y cargo se relacionan con empleado en cardinalidad 1:N; la relación N:M entre empleado y proyecto se resuelve mediante la tabla intermedia asignacion, la cual además almacena las horas asignadas a cada participación.

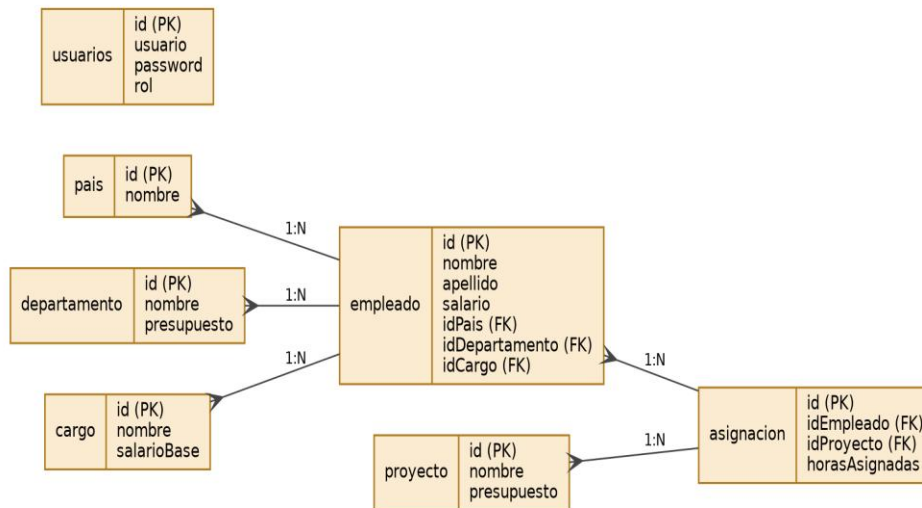


Figura 3. Diagrama entidad-relación de la base de datos SQLite (organizacion.db).

6.4 Estructura del modelo de clases

Clase	Tipo de relación / principio POO aplicado
Persona (abstracta)	Superclase abstracta; define atributos comunes (id, nombre, apellido) y el método abstracto getDescripcion() — base de la herencia y el polimorfismo.
Empleado	Hereda de Persona; se asocia con Pais, Departamento y Cargo; sobrescribe getDescripcion() y toString().
Pais / Departamento / Cargo	Entidades independientes asociadas a Empleado en relación 1:N.
Proyecto	Entidad independiente relacionada con Empleado en N:M a través de Asignacion.
Asignacion	Clase intermedia que resuelve la relación N:M entre Empleado y Proyecto, añadiendo el atributo horasAsignadas.
Empresa	Clase agregadora; aplica composición al contener las listas de Departamentos, Empleados y Proyectos.

6.5 Módulos funcionales de la aplicación

La interfaz gráfica se organiza en módulos independientes, cada uno accesible desde el menú principal de la ventana MDI:

- Inicio de sesión (FrmLogin): autenticación contra la tabla usuarios antes de permitir el acceso al sistema.
- Panel de control (DashboardInterna): muestra indicadores generales (totales de empleados, departamentos, proyectos) y accesos rápidos a los demás módulos.
- Gestión de Países (PaisInterna / PaisControlador / PaisDAO): registro, edición, eliminación y listado de países.
- Gestión de Departamentos (DepartamentoInterna): administración de departamentos y su presupuesto asociado.
- Gestión de Cargos (CargoInterna): administración de cargos y su salario base.
- Gestión de Empleados (EmpleadoInterna): registro de empleados con selección de país, departamento y cargo, y validación de campos obligatorios.
- Gestión de Proyectos (ProyectoInterna): administración de proyectos y su presupuesto.
- Gestión de Asignaciones (AsignacionInterna): asignación de empleados a proyectos con control de horas asignadas, resolviendo la relación N:M.
- Exportación de reportes (ExportadorCSV / ExportadorPDF): exportación del contenido de cualquier tabla del sistema a formato CSV o PDF.

6.6 Persistencia de datos

La conexión hacia SQLite se administra mediante la clase ConexionSQLite, implementada con el patrón Singleton, lo que garantiza una única conexión activa durante toda la ejecución de la aplicación. Al iniciar el programa, esta clase habilita el chequeo de llaves foráneas (PRAGMA foreign_keys = ON, desactivado por

defecto en SQLite) y crea automáticamente el esquema completo de la base de datos si las tablas aún no existen, de modo que el sistema puede ejecutarse sin pasos de configuración manual. Todas las operaciones de acceso a datos utilizan PreparedStatement para prevenir inyección SQL y se agrupan por entidad en las clases del paquete dao.

6.7 Validaciones y manejo de excepciones

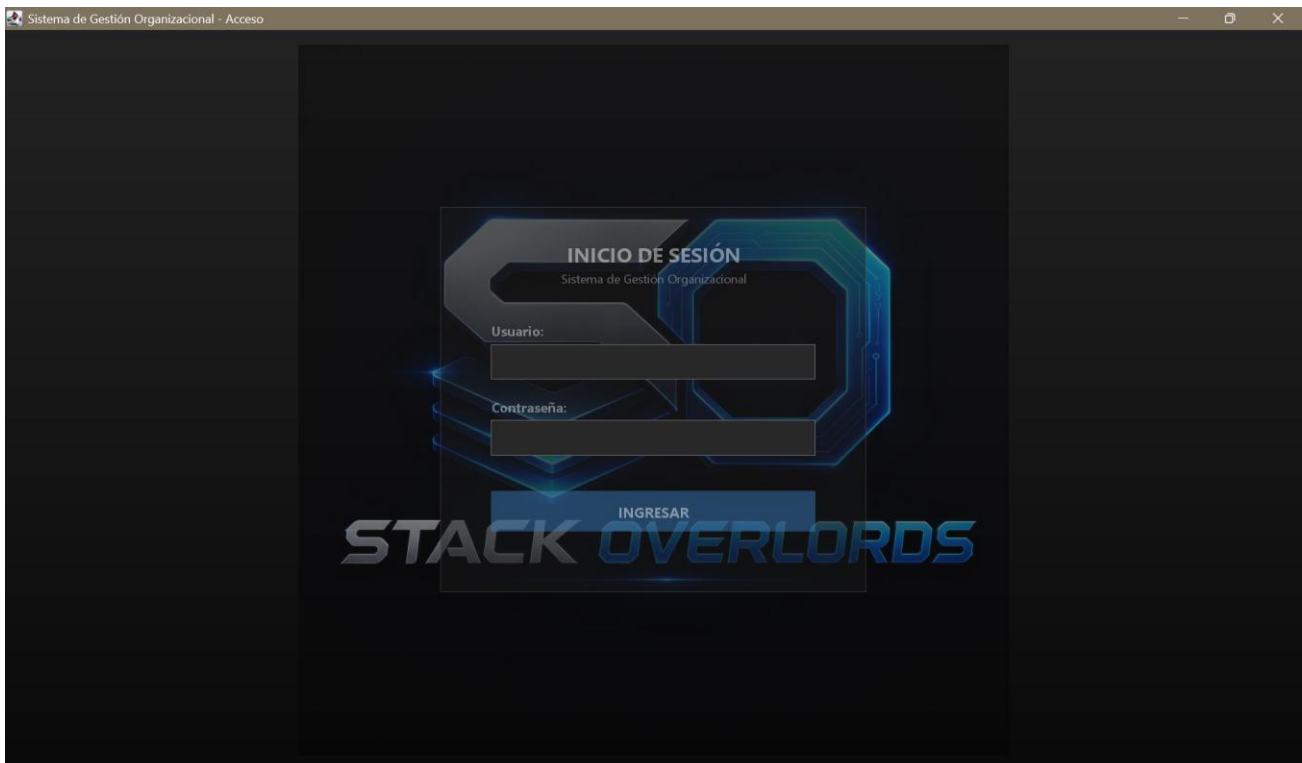
Cada controlador valida los datos de negocio antes de delegarlos al DAO correspondiente (por ejemplo, que el nombre y apellido de un empleado no estén vacíos, que el salario sea mayor que cero, o que se haya seleccionado un país, departamento y cargo válidos). Cuando una validación falla, se lanza una ValidationException con un mensaje descriptivo, la cual es capturada en la capa de vista y mostrada al usuario mediante un cuadro de diálogo, evitando que datos incompletos o inconsistentes lleguen a la base de datos.

7. Resultados obtenidos

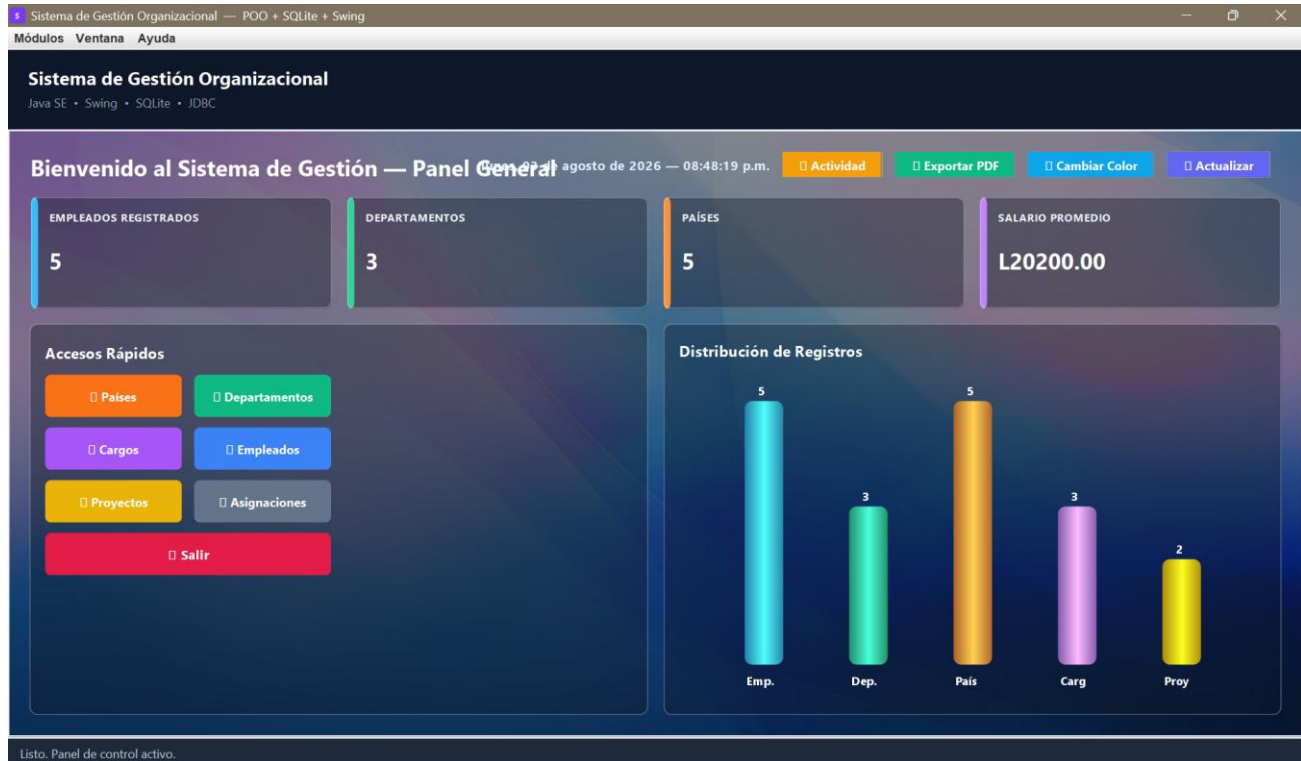
Como resultado del desarrollo se obtuvo una aplicación de escritorio funcional que cumple con los objetivos planteados. A continuación se resumen los principales resultados alcanzados por el equipo:

- Se implementó un modelo de clases completo y coherente, con ocho clases de dominio que aplican de forma verificable encapsulamiento, abstracción, herencia, polimorfismo, asociación y composición.
- Se logró una arquitectura en cinco capas (modelo, dao, controlador, vista, conexion) que separa correctamente las responsabilidades y facilita la extensión del sistema con nuevos módulos.
- Se implementaron operaciones CRUD completas y funcionales para las seis entidades principales del sistema (países, departamentos, cargos, empleados, proyectos y asignaciones), todas con persistencia real en SQLite.
- Se incorporó un módulo de autenticación de usuario y un panel de control (dashboard) con indicadores generales, ampliando el alcance funcional respecto de los requisitos mínimos.
- Se implementó la exportación de reportes a formato CSV y PDF (mediante la librería iText), funcionalidad adicional no contemplada como obligatoria en los requisitos iniciales.
- El sistema mantiene la integridad referencial de la base de datos mediante llaves foráneas, evitando la eliminación de registros que están siendo referenciados por otras tablas.
- Se generó un ejecutable único (.jar) mediante Maven Shade Plugin, que incluye todas las dependencias y permite ejecutar la aplicación sin configuración adicional.

LA VENTANA DE INICIO DE SESIÓN:



EL PANEL DE CONTROL (dashboard):



ALGUNOS MODULOS CRUD:

Módulo de cargos

Sistema de Gestión Organizacional
Java SE • Swing • SQLite • JDBC

Gestión de Cargos

Cargos

Nombre:

Salario base:

Guardar Eliminar Limpiar

ID	Nombre	Salario Base
3	Auxiliar Administrativo	\$15,000.00
5	IT	\$20,000.00
4	Soporte Tecnico	\$14,000.00

Listo. Panel de control activo.

Módulo de empleados:

Sistema de Gestión Organizacional — POO + SQLite + Swing

Módulos Ventana Ayuda

Sistema de Gestión Organizacional

Java SE • Swing • SQLite • JDBC

Gestión de Empleados

Empleados

Nombre: País:

Apellido: Departamento:

Salario: Cargo:

Buscar en tabla:

ID	Nombre	Apellido	Salario	País	Departamento	Cargo
6	Hendric	Castellon	L25,000.00	Nicaragua	Informatica	Soporte Tecnico
5	Audrey	Castro	L20,000.00	Guatemala	Administracion	Auxiliar Administrativo
3	Dany Alberto	Cruz Matute	L16,000.00	Honduras	Informatica	Soporte Tecnico
7	Ricardo	Lagos	L20,000.00	El Salvador	Informatica	IT
4	Jessy	Martinez	L20,000.00	Costa Rica	Administracion	Auxiliar Administrativo

Listo. Panel de control activo.

8. Conclusiones

- El desarrollo del Sistema de Gestión Organizacional permitió aplicar de forma práctica e integrada los principios fundamentales de la Programación Orientada a Objetos, evidenciando su utilidad para modelar problemas reales del ámbito empresarial.
- La separación del sistema en capas (modelo, DAO, controlador y vista) demostró ser clave para distribuir el trabajo entre los ocho integrantes del equipo sin generar conflictos significativos de integración, ya que cada módulo pudo desarrollarse de forma relativamente independiente.
- El uso de una clase abstracta (Persona) y la sobrescritura de métodos en Empleado permitieron comprobar de manera concreta los conceptos de herencia y polimorfismo, más allá de su definición teórica.
- La resolución de la relación muchos a muchos entre Empleado y Proyecto mediante la clase Asignacion reforzó la comprensión de cómo modelar relaciones complejas del mundo real en un diseño orientado a objetos.
- El uso de SQLite junto con JDBC y PreparedStatement brindó una solución de persistencia liviana, portable y suficiente para los fines académicos del proyecto, sin requerir la instalación de un servidor de base de datos independiente.
- El trabajo colaborativo mediante control de versiones fue determinante para integrar los aportes individuales de los ocho integrantes en un producto final coherente y funcional.

9. Recomendaciones

- Incorporar pruebas unitarias automatizadas (por ejemplo, con JUnit) sobre las clases de controlador y DAO, para reducir el riesgo de regresiones ante futuros cambios.
- Evaluar la migración de la interfaz gráfica hacia una tecnología más moderna (JavaFX) o hacia una arquitectura cliente-servidor con interfaz web, si el sistema requiriera en el futuro acceso multiusuario simultáneo.
- Definir roles y permisos diferenciados dentro del módulo de autenticación (por ejemplo, administrador y usuario estándar), en caso de que el sistema se destine a un entorno con múltiples perfiles de usuario.
- Implementar respaldos automáticos periódicos del archivo organizacion.db, dado que SQLite almacena toda la información en un único archivo local.
- Ampliar el módulo de reportes con indicadores estadísticos adicionales (por ejemplo, distribución de empleados por departamento o costos por proyecto), aprovechando la infraestructura de exportación ya existente.
- Mantener actualizada la documentación técnica y los diagramas de clases y entidad-relación conforme el sistema evolucione, para preservar su valor como referencia de diseño.

Anexo A. Instrucciones para la ejecución del sistema

A.1 Requisitos previos

- Java Development Kit (JDK) 17 o superior instalado.
- Apache Maven instalado y configurado en el PATH del sistema (solo necesario para compilar desde el código fuente).
- No se requiere instalar un motor de base de datos aparte: SQLite se ejecuta de forma embebida.

A.2 Ejecución a partir del código fuente

- Clonar o descargar el repositorio del proyecto: [enlace al repositorio Git].
- Abrir una terminal en la carpeta raíz del proyecto (donde se encuentra el archivo pom.xml).
- Ejecutar: `mvn clean package` — esto compila el proyecto y genera un archivo .jar ejecutable con todas las dependencias incluidas (gracias al plugin maven-shade-plugin).
- Ejecutar: `java -jar target/ProyectoFinalPOO.jar` — esto inicia la aplicación.

A.3 Primer inicio de sesión

Al iniciar la aplicación se muestra la ventana de inicio de sesión. El sistema crea automáticamente el esquema de la base de datos y un usuario administrador por defecto:

- Usuario: admin
- Contraseña: 1234

A.4 Uso general

- Desde el panel de control (Dashboard) se pueden abrir los distintos módulos: Países, Departamentos, Cargos, Empleados, Proyectos y Asignaciones.
- Cada módulo permite registrar, modificar, eliminar y listar los registros correspondientes, mostrando los datos en una tabla (JTable).
- Desde los módulos de listado es posible exportar la información visible a formato CSV o PDF mediante los botones de exportación.
- Se recomienda registrar primero países, departamentos y cargos antes de registrar empleados, ya que estos últimos requieren seleccionar dichas entidades.